Listor …

… ett alternativ till arrayer

java.util.ArrayList

Problem med arrayer

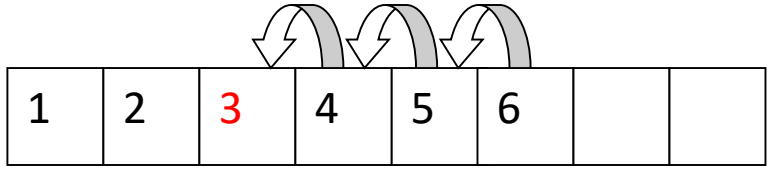
- Arrayer har en fix längd som inte kan ändras
  - Behövs plats för fler element får vi skapa en ny array

```
String [] strings = new String[2];
strings[0] = "One";
strings[1] = "Two";

String[] temp = new String[3];
for(int i = 0; i < strings.length; i++) {
    temp[i] = strings[i];
}
strings = temp; // the first array is garbage collected

strings[2] = "three";
```

- Ta bort och lägga till element
  - Om vi tar bort ett element i en array behöver vi flytta elementen efter detta åt vänster
  - Om vi vill lägga in ett nytt element på givet index måste elementen från denna position åt höger



Listor

- Vi kan naturligtvis skriva kod för att hantera dessa problem med arrayer varje gång det är aktuellt …
- … men bättre att skriva en generell list-klass, som kapslar in en privat array, och hanterar alla dessa problem internt
- Elementen hanteras via publika metoder som add, remove, get, size, ungefär så här …
  - ```
    List list = new List();
    list.add(”one”);
    list.add(”two”);
    list.remove(0); …
    ```
    ```

En enkel list-klass för textsträngar

```
public class SList {

    private String[] arr; // private array
    private int n; // number of elements

    public SList(int capacity) {
        if (capacity <= 0) {
            capacity = 10;
        }
        arr = new String[capacity];
        n = 0;
    }

    public int size() { return n; }

    public String get(int index) {
       if(index < 0 || index >= n) throw new IndexOutOfBoundsException(index);
       return arr[index];
    }
```

En (enkel) list-klass för textsträngar

```
public class SList {
    …
    public void addLast(String s) {
        if (arr.length <= n) {
            resize();
        }
        arr[n++] = s;
    }

    public String remove(int index) {
        String s = arr[index];
        pack(index);
        return s;
    }
    …
```

En (enkel) list-klass för textsträngar

```
public class List {
    …
    private void resize() {
        String[] tmp = new String[arr.length * 2];
        for (int i = 0; i < arr.length; i++) {
            tmp[i] = arr[i];
        }
        arr = tmp;
    }

    private void pack(int index) {
        for (int i = index; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;
    }
    …
```

java.util.ArrayList

- Naturligtvis finns redan klasser för listor i Javas API
  - ArrayList, LinkedList, Vector, …
- Klasserna är ”generiska”, dvs. vi kan ange vilken typ av objekt som ska lagras i listan
  - Typparameter <T>
  - ArrayList<String> strings = new ArrayList<String>();
- Använd lista istället för arrayer när
  - antal element varierar
  - element ofta läggs till eller tas bort

ArrayList - exempel

```
ArrayList<String> strings = new ArrayList<>();
strings.add(new String("Lindström"));
strings.add(new String("Cajander"));

String s = strings.get(1); // second element, ”Cajander”

for (int i = 0; i < strings.size(); i++) {
    System.out.println(strings.get(i).toString());
}

for (String str : strings) {
    System.out.println(str);
}
```

ArrayList

- Dokumentation: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html

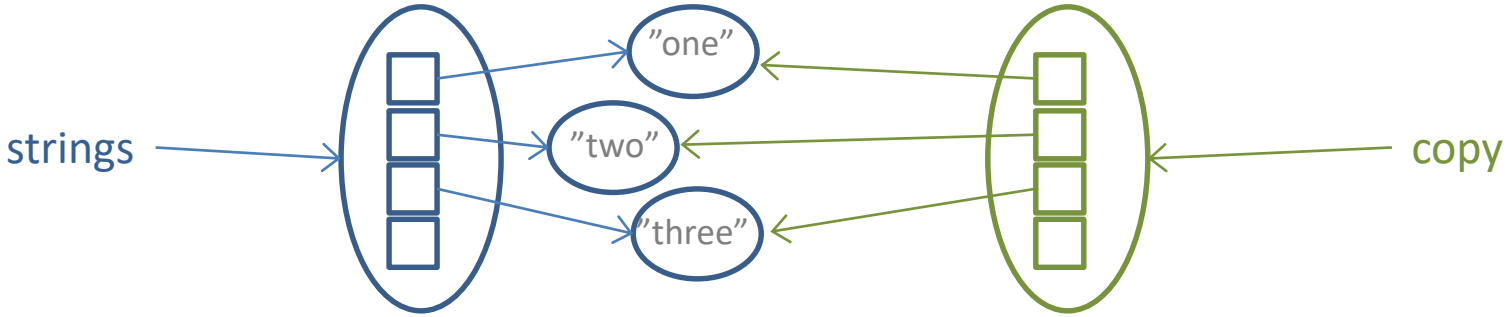
- ArrayList<*Typ*>() – konstruktor som skapar en lista, initialt med kapacitet för 10 objekt
- add(*Typ* obj) – lägger till ett objekt sist i listan
- add(int index , Typ obj) – flyttar elementen från angiven position ett steg framåt och placerar sedan in obj på angiven position
- int size() – returnerar antalet objekt i samlingen
- *Typ* get(int index) – returnerar objektet i angiven position
- *Typ* remove(int index) – tar bort objektet på angivet index. Listan packas automatiskt.
- boolean remove(*Typ* obj) – tar bort objektet (om det finns i listan)
- clear() – tömmer listan
- *Typ*[] toArray() – returnerar en array (kopia) med referenser till objekten i listan

Skapa en kopia av en lista (ArrayList)

- Det finns flera situationer då vi behöver en kopia av en lista …
  - … t.ex. då en metod ska returnera en intern, och privat, lista
- Alternativ 1, använd en konstruktor i ArrayList
  - ```
    ArrayList<String> strings = new ArrayList<>();
    strings.add(“one”); …
    ArrayList<String> copy = new ArrayList<>(strings);
    ```
- Alternativ 2, använd metoden clone
  - ```
    ArrayList<String> strings = new ArrayList<>();
    strings.add(“one”); …
    ArrayList<String> copy = (ArrayList<String>) strings.clone();
    ```

Skapa en kopia av en lista (ArrayList)

- I båda fallen fås en “grund kopia”, shallow copy



- Vi får ett nytt list-objekt, men referenserna i den interna arrayen refererar samma objekt som det ursprungliga list-objektet